Task 3: Ranking Development

I built a multi-algorithm framework (rankers.py) with BM25, BM25+, TF-IDF cosine, Hybrid, and Semantic reranking. All methods used fixed hyperparameters and deterministic tie-breaking, candidate TF and lengths were precomputed for efficiency.

Method	r
BM25	0.3828
BM25+	0.6353
TF-IDF	0.5386
Hybrid	**0.6556**
Semantic	0.6348

Table 3.1 Dev results (Pearson r)

Analysis: BM25+ improved over classic BM25 by flooring negative IDF and tuning (k1=1.4, b=0.7), stabilizing scores for frequent terms. TF-IDF provided orthogonal evidence but underperformed BM25+ individually. Hybrid, a weighted blend (0.8 BM25+, 0.2 TF-IDF) with min–max normalization, achieved the best correlation (r=0.6556), validating our hypothesis that combining complementary signals improves robustness. Semantic reranking (BM25+ shortlist of ≤20 docs + hash-cosine) was competitive but did not surpass Hybrid, likely due to noisy approximations for short queries.

Hybrid was chosen as **default** because it consistently gave the highest dev correlation under fixed hyperparameters. **Trade-offs:** BM25 can over-penalize frequent terms, TF-IDF struggles on short or mismatched queries, and Semantic reranking adds noise for small inputs; Hybrid offsets these issues but with slight normalization overhead.

Task 4: System Optimizations (excluding ranking)

I implemented the end-to-end pipeline in system/search_system.py and evaluated runs with metrics/eval_map.py.Default run (run_default.json) achieved:

Run	MAP@10	MRR@10	#Q	covQ
run_default.json	0.5888	0.8770	21	21

Table 4.1 Run results

Optimization 1:

Robust input handling. I added encoding-safe JSON/JSONL loaders with fallbacks (utf-8, cp1252, gzip). Hypothesis: preventing document/query drops ensures consistent coverage and avoids MAP penalties from missing data. Implemented in _iter_lines_flex and _open_text_flex. Result: full coverage (covQ=21/21) was achieved, ensuring reliable evaluation.

Optimization 2:

Candidate determinism. I enforced sorted candidate doc IDs before ranking, and packaged only the top-10 results per query. Hypothesis: deterministic ordering prevents evaluation noise and guarantees reproducibility. Implemented immediately before rank_documents. Result: stable MAP/MRR across repeated runs, critical for fair comparisons.

These system optimizations improved reliability rather than raw MAP, but were essential for efficiency and correctness: indexing is built once per run in .cache, candidate sets are bounded, and evaluation operates on consistent inputs. **Efficiency Trade-off:** Robust loaders and deterministic candidate handling add minor I/O and sorting overhead, but they ensure full query coverage and reproducible MAP/MRR results.

Declaration

I acknowledge that I used ChatGPT to assist in drafting and refining the wording of this write-up. All experiments, implementation work, analysis, and results presented here are entirely my own and reflect my independent effort.